SentinelWatch

Final-Year Project Review Package

Architecture, implementation review, diagrams, presentation guide, viva questions, demonstration guide, and completion checklist

Prepared from the inspected SentinelWatch repository. All claims distinguish verified behaviour from unconfigured or unverified integrations.

Contents

1. Project Review
2. Presentation and Speaking Guide
3. Viva Questions and Answers
4. Live Demonstration Guide
5. Completion Checklist
6. Diagram Sources

How to use this PDF: read the project review first, rehearse with the speaking guide, practise the viva answers, then follow the demo guide on review day. The final sections preserve the maintainable Mermaid diagram source so diagrams can be rendered later without re-creating them.

SentinelWatch Project Review

Evidence baseline: source inspected and local application verified on 12 October 2026. The dashboard was running at `http://127.0.0.1:8001`; health returned `200 OK`; `pytest -q` reported 19 passed and `ruff check .` passed. This is not proof of a complete operational satellite-analysis workflow.

1. Project Overview

Title: SentinelWatch — AI-assisted satellite land-change monitoring. Domain: remote sensing, GIS, machine learning, and web information systems.

Problem. Large areas are difficult to inspect repeatedly on the ground. Vegetation loss, excavation, construction and other physical land-surface changes can go unnoticed for weeks. SentinelWatch helps an operator define an Area of Interest (AOI), find Sentinel-2 observations for it, compare two dates when the required services are configured, and place potential changes in a review queue.

Important boundary: it detects *possible physical change*, not crime. An alert is a lead requiring imagery-quality, seasonality, permits, and local verification. It cannot legally prove illegal cutting, mining, or construction.

Motivation and proposed solution. Instead of manually browsing image catalogues, the system keeps AOIs, searches the official Copernicus catalogue, records provenance, preprocesses suitable image pairs, runs a baseline or optional model, calculates change regions, and creates reviewable cases/alerts. Its intended users are forest, planning, environment, or research reviewers. Benefits are a repeatable evidence trail and a way to prioritize human attention.

Objectives and scope. The implemented scope is a FastAPI dashboard, SQLite persistence, CDSE catalogue adapter, monitoring-run orchestration, an optional GeoTIFF path, baseline change detection, optional Siamese U-Net code, alert/case workflow, department routing, notification attempt records, PDF case-report code, training/evaluation scripts, and tests. It is not yet a validated production surveillance service: no trained checkpoint, credentials, Rasterio processing runtime, SMTP delivery, or real end-to-end analysis result was verified in this environment.

Key assumptions/limits. An AOI must be supplied; a suitable cloud-limited observation pair must exist; authenticated imagery access and Rasterio are needed for operational raster processing; any model must have a lawful labelled dataset and validated checkpoint. Clouds, shadows, seasonal growth, imperfect alignment and resolution can resemble change.

30-second explanation. “SentinelWatch is a decision-support web application. A user marks an area, the system searches official Sentinel-2 satellite metadata, and—when imagery services are configured—compares two dates for potential land change. It records the evidence and raises a review alert. It never claims that a change is automatically illegal.”

One-minute explanation. “The project combines a browser dashboard, a Python FastAPI backend, SQLite database and Copernicus Data Space catalogue. Areas and monitoring runs are stored locally. The backend finds observation metadata, chooses a before/after pair, then can retrieve six-band imagery, mask poor pixels, calculate spectral/NDVI differences and extract changed regions. A Siamese U-Net is available in code but there is no trained checkpoint installed, so the honest present state is a labelled baseline path plus model-training scaffolding. Findings can become cases, alerts, department assignments and a PDF report.”

Two-minute explanation. “The practical problem is that field inspection cannot cover every large or remote area. SentinelWatch therefore starts with a specific AOI and dates rather than making a vague accusation. It uses the CDSE STAC catalogue to discover official Sentinel-2 products. Metadata such as product ID, acquisition time and cloud cover is stored so a reviewer can trace where evidence came from. If suitable images and configuration are available, the backend requests imagery, aligns the rasters, masks cloud/shadow pixels, compares vegetation and visible-band signals, cleans the change mask, and turns connected changed pixels into regions with physical area. It can label a region as vegetation loss, construction-like change or land disturbance; that label is a screening classification, not a legal conclusion. Findings, cases, alerts and reviewer notes are persisted. The dashboard makes limitations visible: this local run has no image credentials, Rasterio or validated model, so it shows those blockers instead of producing invented results. Unit tests verify many code contracts, but a live data-to-alert demonstration still requires configuration and real imagery.”

2. System Architecture

!As-built architecture source (diagrams/architecture.mmd)

The frontend is a static Jinja-served HTML/CSS/JavaScript dashboard using Leaflet when its CDN assets load. It calls JSON endpoints in the FastAPI API layer. Routers in `app/api/monitoring.py` and `app/api/operations.py` validate requests and write through SQLAlchemy into the local SQLite database (`instance/sentinelwatch.db`). `app/services/monitoring.py` executes a queued run in a FastAPI background task (and contains optional scheduler logic); it searches CDSE metadata, stores refresh/acquisition records and only moves to imagery processing if prerequisites exist.

`app/satellite/copernicus.py` is the external adapter: STAC metadata search is public; Process API raster retrieval requires OAuth credentials. `app/services/geospatial.py` optionally needs Rasterio/PyProj to align, mask and calculate spatial output. `app/services/analysis.py` uses the numpy/Pillow baseline or `app/ml/inference.py` only when a loadable model checkpoint exists. Results become database analysis, case, finding and alert records. `app/services/alerting.py`, `notifications.py`, and `reporting.py` respectively create review alerts, record/attempt optional SMTP delivery, and generate a case PDF when ReportLab is available.

Errors are represented as failed/blocked monitoring states and provider-refresh records rather than fabricated imagery. Actual architecture differs from the target: the target includes credentials, Rasterio, a validated checkpoint, reliable background worker/scheduler and SMTP; the inspected local run lacks all of those operational prerequisites.

3. Module Description

Module: Dashboard
Purpose: Operate and view status
Inputs: Browser actions
Processing: Fetch/render JSON
Outputs: AOI/case/alert views
Main files/classes/APIs: `templates/index.html`, `static/app.js`
Current status: Implemented and manually opened

Module: AOI management
Purpose: Persist monitoring areas
Inputs: geometry/name
Processing: validation, pause/resume
Outputs: area record
Main files/classes/APIs: `api/monitoring.py /areas`
Current status: Implemented and tested

Module: Priority discovery
Purpose: Suggest grid cells with catalogue availability
Inputs: region/bbox
Processing: STAC per cell and heuristic score
Outputs: preliminary candidates
Main files/classes/APIs: `priority_discovery.py`, `/priority-candidates`
Current status: Implemented and tested with mock provider; not change detection

Module: Catalogue search
Purpose: Find official product metadata
Inputs: AOI/time/cloud filters
Processing: CDSE STAC POST search
Outputs: acquisitions
Main files/classes/APIs: `satellite/copernicus.py`
Current status: Implemented; live metadata query verified

Module: Monitoring orchestration
Purpose: Run the workflow
Inputs: area/run ID
Processing: metadata, pair choice, state/audit
Outputs: monitoring status
Main files/classes/APIs: `services/monitoring.py`
Current status: Implemented and unit-tested; full path unverified

Module: Geospatial preparation
Purpose: Make rasters comparable
Inputs: downloaded bands
Processing: align, reproject, masks, NDVI
Outputs: arrays/metadata
Main files/classes/APIs: `services/geospatial.py`
Current status: Implemented but not verified locally (Rasterio absent)

Module: Change analysis
Purpose: Locate candidate change
Inputs: before/after arrays
Processing: baseline or optional model, components
Outputs: findings
Main files/classes/APIs: `services/analysis.py`
Current status: Baseline unit-tested; live imagery path unverified

Module: Siamese model
Purpose: Learned binary change map
Inputs: paired tensors
Processing: shared encoder U-Net inference
Outputs: probability mask
Main files/classes/APIs: `ml/siamese_unet.py`, `inference.py`
Current status: Partially implemented; no checkpoint/evaluation

Module: Alerts/routing
Purpose: Prioritize human review
Inputs: findings/department
Processing: threshold, dedup, notification attempt
Outputs: alert/status
Main files/classes/APIs: `alerting.py`, `notifications.py`
Current status: Implemented and tested; SMTP not configured

Module: Persistence
Purpose: Evidence trail
Inputs: all records
Processing: SQLAlchemy relations
Outputs: SQLite records
Main files/classes/APIs: `models.py`, `migrations.py`
Current status: Implemented and tested

Module: Reporting
Purpose: Produce case PDF
Inputs: stored case
Processing: ReportLab document
Outputs: PDF response
Main files/classes/APIs: `reporting.py`
Current status: Implemented but live PDF generation not verified

Module: Training/evaluation
Purpose: Train/measure a model
Inputs: labelled dataset
Processing: BCE/Dice training, metrics
Outputs: checkpoint/metrics
Main files/classes/APIs: `training/`
Current status: Scaffolding only; no dataset/results

Beginner view of the main modules. The dashboard is the control room. AOI management tells the system where to look. The catalogue adapter is a library search: it finds product descriptions, not automatically downloaded pixels. The monitoring service coordinates steps. Geospatial preparation makes two images line up and removes unreliable pixels. Analysis highlights suspicious pixels; alerting puts meaningful findings in a human queue. The database remembers every item, while reports make a case easy to share.

4. Data Flow Diagrams

Sources: Level 0 (diagrams/dfd-level-0.mmd), Level 1 (diagrams/dfd-level-1.mmd), Level 2 (diagrams/dfd-level-2.mmd).

Level 0. The operator sends AOIs and review actions to SentinelWatch. SentinelWatch queries CDSE and returns dashboard views/reports. Optional email is an outbound side effect only after configuration. Arrows mean data requests/responses, not proof that every external service is active.

Level 1. Process 1 stores areas/candidates in the SQLite data store. Processes 2–4 create a run, search metadata and choose dates. Processes 5–6 occur only with imagery prerequisites. Process 7 stores review work; process 8 presents records. The database is the only shown internal data store.

Level 2. The key workflow persists catalogue metadata, selects a temporal pair, checks prerequisites, requests imagery, preprocesses it, then uses either validated-model inference or the baseline. Thresholding and connected components turn a pixel mask into a physical finding. If prerequisites are absent, the run is blocked/failed transparently instead of pretending it analysed imagery.

5. UML Diagrams

Maintained Mermaid sources: use case (diagrams/use-case.mmd), activity (diagrams/activity.mmd), sequence (diagrams/sequence.mmd), component (diagrams/component.mmd), deployment (diagrams/deployment.mmd), and ER diagram (diagrams/er-diagram.mmd). The diagrams use Mermaid flowchart/sequence/ER grammar and were manually checked against routes/models. Mermaid CLI was not installed, so no rendered image files are claimed.

The use-case diagram separates the operator from CDSE and optional SMTP. The activity diagram makes the prerequisite branches visible. The sequence diagram shows `202 Accepted` being returned while the monitoring service continues in the background. The component/deployment diagrams distinguish local host components from external CDSE/SMTP. The ER diagram reflects actual principal SQLAlchemy models; minor helper/audit fields are deliberately omitted for slide readability.

6. Technologies and Tools Used

Technology/tool	Actual use in SentinelWatch	Why it is used	Evidence in repository
Python	application and ML/training code	concise backend/scientific ecosystem	`app/`, `training/`
FastAPI + Uvicorn	HTTP app and API server	typed routes/OpenAPI/dev server	`app/main.py`, `requirements-core.txt`
Jinja2 + HTML/CSS/JS	single-page dashboard shell	lightweight UI without a frontend build	`templates/`, `static/`
Leaflet	map in AOI UI	browser mapping/drawing context	`index.html`, `app.js`
SQLite + SQLAlchemy	persistent local data	simple demonstrable relational storage	`database.py`, `models.py`
HTTPX	CDSE HTTP calls	async-friendly HTTP client	`satellite/copernicus.py`
Copernicus Data Space STAC	product metadata search	official Sentinel catalogue	`copernicus.py`

Technology/tool: NumPy + Pillow
Actual use in SentinelWatch: baseline array/image processing
Why it is used: portable numerical/image operations
Evidence in repository: `analysis.py`, core requirements

Technology/tool: Rasterio/PyProj
Actual use in SentinelWatch: optional spatial raster work
Why it is used: alignment, georeferencing and pixel area
Evidence in repository: operational requirements, `geospatial.py`

Technology/tool: PyTorch
Actual use in SentinelWatch: optional Siamese U-Net
Why it is used: learned paired-image model
Evidence in repository: `ml/`, ML requirements

Technology/tool: ReportLab
Actual use in SentinelWatch: optional case PDF
Why it is used: server-side printable report
Evidence in repository: `reporting.py`

Technology/tool: pytest + Ruff
Actual use in SentinelWatch: tests and lint
Why it is used: behaviour checking/code quality
Evidence in repository: `tests/`, dev requirements

Why these choices: FastAPI makes APIs and OpenAPI documentation quick to expose; SQLite reduces setup for a final-year demo; SQLAlchemy gives structured persistence; STAC is a standard catalogue interface; NumPy enables a baseline even without a trained model; PyTorch keeps a credible upgrade path. “Listed in requirements” does not mean “active locally”: Rasterio/PyTorch/ReportLab are optional dependency groups and the current system status says Rasterio and model checkpoint are absent.

7. Hardware and Software Requirements

Scenario: Dashboard/API
Practical requirement: 64-bit Windows/Linux/macOS, Python compatible with project dependencies, 4 GB RAM minimum, modern Chrome/Edge/Firefox, core requirements, local write permission for `instance/` and `storage/`

Scenario: Real metadata search
Practical requirement: internet access to CDSE STAC; no OAuth credential required for the observed metadata query

Scenario: Imagery/inference
Practical requirement: operational requirements, CDSE OAuth credentials, Rasterio/PyProj, disk space for GeoTIFFs; 8 GB RAM recommended

Scenario: Model training
Practical requirement: ML requirements, labelled lawful paired dataset, substantially more storage/RAM; GPU is optional but recommended for practical training, not required by source

Use the exact dependency files: `requirements-core.txt`, `requirements-operational.txt`, `requirements-ml.txt`, and `requirements-dev.txt`. Sentinel-2 rasters can be large; keep the storage directory outside version control and budget tens of GB for repeated AOIs. Leaflet/CDN map tiles and CDSE calls need internet.

8. Algorithms and Methodology

Candidate discovery. `services/priority_discovery.py` grids a user region into 1–3 cells each dimension, searches catalogue availability, then uses `100 × (0.45 freshness + 0.35 cloud_quality + 0.20 availability)`. It ranks *where data is available*, not where illegal activity occurred. Mock-provider tests cover persistence/approval.

Observation search/pairing. `CopernicusDataSpaceProvider.search_acquisitions` sends a STAC search and parses product ID, date and cloud cover. The monitoring service chooses time-separated suitable records. Acquisition date is when the satellite observed the Earth; retrieval date is when the application downloaded/asked for the product. A catalogue is the external product index; SQLite is SentinelWatch’s local evidence database.

Preprocessing and NDVI. `geospatial.py` aligns/reprojects rasters, uses SCL/CLM masks and calculates NDVI when red/NIR bands exist:

```
NDVI = (NIR − Red) / (NIR + Red)
```

NDVI measures vegetation greenness but is vulnerable to cloud, shadow, season, sensor geometry and alignment differences. This code is present but unverified in the local runtime because Rasterio is absent.

Baseline change detection. `analyse_arrays` normalises illumination, computes RGB and vegetation/NDVI change probability, thresholds and morphologically cleans the mask, finds connected components, labels each region and converts pixel count using real pixel area. This is image differencing: hand-designed spectral rules, not learned classification. Controlled array tests exercise it.

Optional Siamese U-Net. `SiameseUNet` uses shared-encoder before/after inputs and produces a change-probability map. `inference.py` loads it only if a checkpoint is available. Training code uses paired dataset loading and BCE/Dice-style loss/evaluation utilities. No checkpoint, dataset, held-out metric, accuracy/IoU/F1, or live inference evidence exists; do not quote performance numbers.

Alert priority/deduplication. The alert service derives a heuristic score from confidence, persistence and area; it is not a calibrated probability. Alerts are created for configured thresholds and duplicate review records are avoided through case/finding state. SMTP attempts are always recorded, including `NOT_CONFIGURED`, so delivery is not falsely asserted.

9. Implementation Details

```
app/              FastAPI, routes, services, satellite and ML code
training/         dataset, training, evaluation and registration utilities
tests/            API/service/analysis contracts
instance/         local SQLite runtime database
storage/          generated previews/reports/runtime artifacts
```

`app/main.py` creates the app, static/template routes, lifecycle initialization and routers. Important endpoints include `POST/GET /api/areas`, `POST /api/areas/{id}/check-now`, `GET /api/system/status`, `POST /api/priority-candidates/discover`, `POST /api/alerts/{id}/assign`, `GET /api/data-status`, plus case/alert/report routes in the main app. The frontend uses `fetch` to call these endpoints and renders the returned JSON.

The core monitoring function is `execute_monitoring_run(run_id)` in `app/services/monitoring.py`; it persists status at each step. The implementation follows this shape:

```
# app/services/monitoring.py (conceptual excerpt of actual flow)
acquisitions = provider.search_acquisitions(...)
pair = provider.select_pair(acquisitions, ...)
# only then, with credentials and Rasterio available:
analysis = analyse_arrays(before, after, ...)
```

Configuration is in `app/config.py` and `.env.example`: database/storage paths, CDSE URLs/OAuth, cloud/lookback/pixel controls, model location, scheduler, demo mode and SMTP. Never commit the real `.env`. `migrations.py` applies lightweight additive SQLite schema updates; it is useful for this project but is not a full production migration system such as Alembic.

To install and run from the repository:

```
python -m pip install -r requirements-core.txt
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Visit `http://127.0.0.1:8000/` and `http://127.0.0.1:8000/docs`. Add optional dependency sets only when their prerequisite is needed. Detailed repeatable steps are in the Demo Guide (SentinelWatch_Demo_Guide.md).

10. Work Completed So Far

Feature: Dashboard/health
Evidence found: live page and `/api/health`
Testing performed: manually opened, API health
Status: Implemented and tested
Remaining work: add real operational records

Feature: SQLite models/migration
Evidence found: models + database file
Testing performed: API/service tests
Status: Implemented and tested
Remaining work: production migration strategy

Feature: CDSE metadata search
Evidence found: real adapter and a live 10-result query
Testing performed: live metadata query + mocks
Status: Implemented and tested for metadata
Remaining work: retain proof after final setup

Feature: AOI/run API
Evidence found: routes/background workflow
Testing performed: API tests
Status: Implemented and tested
Remaining work: live configured run

Feature: Baseline analysis
Evidence found: numpy code
Testing performed: controlled array tests
Status: Implemented and tested
Remaining work: validate on labelled imagery

Feature: Raster/imagery path
Evidence found: source code
Testing performed: no local E2E
Status: Implemented but not fully tested
Remaining work: credentials/Rasterio/live pair

Feature: ML model
Evidence found: architecture/training scripts
Testing performed: no checkpoint/evaluation
Status: Partially implemented
Remaining work: dataset, train, register, validate

Feature: Alerts/routing
Evidence found: services/routes
Testing performed: tests with controlled state
Status: Implemented and tested
Remaining work: real alert from satellite run

Feature: SMTP
Evidence found: code/status record
Testing performed: config-not-ready path
Status: Implemented but not fully tested
Remaining work: SMTP configuration/delivery test

Feature: PDF report
Evidence found: ReportLab service/route
Testing performed: route contract test
Status: Implemented but not fully tested
Remaining work: generate/open real PDF

Feature: Synthetic seed
Evidence found: `demo_seed.py`
Testing performed: not used in this run
Status: Demo functionality
Remaining work: label clearly if used

11. Current Project Status and Progress

Verified now: dashboard, health/status API, SQLite connection, primary API/service tests, Ruff, real STAC *metadata* retrieval. The local dashboard truthfully reports 0 AOIs, 0 observations, 0 analyses and 0 alerts; that is an empty real database, not a failure of the page.

Waiting/blocked: authenticated Process API imagery, Rasterio preprocessing, live change result, trained model, SMTP delivery and real PDF inspection. The database itself is connected and persists to the SQLite file. Scheduler is disabled. No major failing unit-test bug was found in this review, but live end-to-end status remains unverified.

Priority list. P0: configure lawful CDSE credentials and operational dependencies; create a real AOI and demonstrate a full metadata-to-pair run. P1: install Rasterio, obtain/cache a suitable cloud-limited image pair, validate preprocessing/baseline output, capture a genuine case/alert/PDF. P2: prepare labelled data, train/evaluate/register model and report measured metrics; configure/test SMTP. P3: durable worker queue, authentication/roles, PostGIS/object storage, model monitoring and a full migration framework.

No completion percentage is assigned because operational work is uneven: many code modules exist, but the critical live imagery/model path has not been verified.

12. Results and Screenshots of Implemented Modules

An actual live dashboard was inspected on 12 October 2026. It showed database connected, but 0 stored observations/areas/analyses/alerts; CDSE imagery credentials, Rasterio and validated model were explicitly unavailable. This is useful evidence of honest state reporting, not a satellite-analysis result. The screenshot-capture interface available during this review could display the live dashboard but did not export a repository image file; therefore `docs/screenshots/` is present but contains no invented image.

Figure: 1
Module: Main dashboard
Evidence/caption: Live page verified: health/status cards and configured limitations displayed
What to say: “This is the real dashboard. It shows connected SQLite but no fabricated observation or alert.”

Figure: 2
Module: Priority discovery
Evidence/caption: UI route exists; no live candidate is stored
What to say: “Candidates are preliminary catalogue-availability cells, not detected incidents.”

Figure: 3
Module: Change result
Evidence/caption: Not captured: no credentials/Rasterio/pair
What to say: “I will capture this only after a successful real run.”

Figure: 4
Module: Alert/department/report
Evidence/caption: Not captured: no real finding/case
What to say: “The API and tests exist; the operational artifact still needs verification.”

Actual safe response evidence: `GET /api/health` returned `{"status":"ok","service":"SentinelWatch"}`. A live CDSE metadata search over a small Bengaluru-area bbox with 365-day lookback and cloud ≤80 returned 10 products; newest recorded result was `S2B_MSIL2A_20261009T050719_N0513_R019_T43PGQ_20261009T090953` acquired `2026-10-09T05:07:19.024000`, cloud `57.33`. This confirms catalogue metadata access only, not image download or detection.

13. Testing and Results

Test ID: T01
Component: API health/dashboard
Scenario: app starts and health route responds
Expected result: 200 service status
Actual result: 200 `SentinelWatch`
Status: Passed

Test ID: T02
Component: Analysis
Scenario: controlled before/after arrays
Expected result: identify synthetic change safely
Actual result: suite passed
Status: Passed

Test ID: T03
Component: Provider
Scenario: STAC parsing/pair selection with mocks
Expected result: correct metadata/pair logic
Actual result: suite passed
Status: Passed

Test ID: T04
Component: Monitoring service
Scenario: metadata persistence and truthful blocked states
Expected result: status/evidence persisted
Actual result: suite passed
Status: Passed

Test ID: T05
Component: Operations
Scenario: candidate/department/assignment APIs
Expected result: stored outputs and notifications
Actual result: suite passed
Status: Passed

Test ID: T06
Component: Reporting
Scenario: report endpoint contract
Expected result: expected response/fallback contract
Actual result: suite passed
Status: Passed

Test ID: T07
Component: Lint
Scenario: `ruff check .`
Expected result: no lint errors
Actual result: “All checks passed”
Status: Passed

Test ID: T08
Component: Live CDSE metadata
Scenario: public STAC query
Expected result: products returned
Actual result: 10 products returned
Status: Passed

Test ID: T09
Component: Live imagery-to-alert
Scenario: credentials/raster/model workflow
Expected result: real case/alert
Actual result: Not run; prerequisites missing
Status: Not run

Test ID: T10
Component: SMTP/PDF artifact
Scenario: external delivery/open generated PDF
Expected result: verifiable delivery/artifact
Actual result: Not run
Status: Not run

Command results were `19 passed, 103 warnings in 5.82s` and `All checks passed!`. The warnings were deprecation warnings from dependencies, not failing tests. A unit test proves an isolated behaviour; it does not prove CDSE retrieval, raster alignment, model quality or email delivery in production.

14. Future Work and Remaining Tasks

Incomplete before final submission: install operational dependencies; configure CDSE OAuth; demonstrate one lawful AOI with a suitable pair; inspect the generated analysis/case/alert/report; capture the missing real screenshots. Train and hold-out evaluate the Siamese U-Net on a documented lawful dataset before enabling it. Report only measured metrics.

Improvements: better cloud/shadow/seasonality controls, parameter calibration against labelled data, explicit human verification workflow, SMTP configuration, scheduled worker with durable queue, authentication/audit permissions, and a stronger migration strategy.

Long-term: PostGIS/object storage, multiple satellite sources, change-history timelines, model drift evaluation, field-validation feedback loop, and scalable deployment. None of these should be presented as currently complete.

15. Conclusion

SentinelWatch addresses the practical challenge of repeatedly screening remote areas by combining an AOI dashboard, official Sentinel-2 catalogue search, local evidence persistence and a proposed raster/model analysis pipeline. The verified capabilities are the web/API/data foundation, metadata search, baseline-analysis code contracts, alert workflow contracts and transparent health/model status. Its value is prioritizing human investigation, not declaring illegality. The main next step is a fully configured, documented real image-pair run followed by honest validation of the baseline and—only later—the trained model.

SentinelWatch Review Speaking Guide

Explain it at three levels

Beginner level. Think of SentinelWatch as a watchtower, not a judge. I mark a place on a map. The system looks for satellite observations at two dates and, if it has the required image-processing setup, highlights land that looks different. A human still checks whether it is real and whether it is legal.

Technical level. The browser dashboard calls FastAPI endpoints. SQLAlchemy stores AOIs, acquisitions, monitoring runs, analyses, cases, findings, alerts and review actions in SQLite. A CDSE STAC adapter searches metadata. The optional downstream path retrieves raster bands, aligns/masks them, computes spectral/NDVI change, creates connected regions and stores findings. A PyTorch Siamese U-Net is coded but no validated checkpoint is installed.

Review-panel level. I chose a simple web/API/database structure so the evidence trail is inspectable and testable. I deliberately show blocked states instead of faking outputs. Code and unit tests verify many modules; the final operational proof still needs credentials, Rasterio, an image pair and a lawful labelled dataset/model evaluation.

8–10 minute slide plan

1. SentinelWatch: title and idea

- AI-assisted satellite land-change monitoring.
- Decision support, not automatic illegality detection.
- Show: dashboard title/architecture diagram.
- Say: “My project helps a reviewer focus on possible changes. The key word is possible; it creates a lead, not a legal verdict.”
- Transition: “Why do we need such a system?”
- Panel may ask: What is the exact problem? Answer: large/remote areas cannot be manually inspected frequently.

2. Problem and motivation

- Deforestation, land disturbance and construction can be spatially large and time-dependent.
- Ground verification is still required.
- Show: a simple before/after concept (not a fabricated project result).
- Say: “The goal is to reduce the search area for a human inspector.”
- Transition: “I designed the system around a traceable workflow.”
- Question: Can it prove crime? No; imagery alone cannot establish legality or permissions.

3. Objectives and scope

- AOI management, official catalogue search, evidence persistence, optional analysis, alerts/reports.
- Explain current boundary: model and live raster path not validated locally.
- Show: completion checklist.
- Say: “I separate code that exists from functionality proven in this environment.”
- Transition: “Here is the actual architecture.”

4. As-built architecture

- Browser → FastAPI → SQLite; monitoring service → CDSE; optional Rasterio/model/SMTP.
- Show: `architecture.mmd` rendered in Mermaid viewer.
- Say: “The browser never talks directly to the database or CDSE; the backend controls validation and provenance.”

- Transition: “Now I will break this into modules.”
- Question: Why SQLite? Simple persistent local database for a final-year deployment; PostgreSQL/PostGIS is a future scale option.

5. Modules and workflow

- AOIs, candidate discovery, metadata search, monitoring, analysis, alerting, reporting.
- Show: DFD Level 1.
- Say: “The candidate module only ranks catalogue availability; it is not a detector.”
- Transition: “The detailed workflow shows important failure branches.”
- Question: What happens if no suitable pair exists? The run records that state; no alert should be invented.

6. Data flow and UML

- Context, detailed comparison, sequence/ER views.
- Show: DFD Level 2 and ER diagram.
- Say: “The ER diagram shows how a monitoring area leads to runs, analyses, cases, findings and alerts.”
- Transition: “Next are the actual algorithms.”
- Question: Catalogue vs database? Catalogue is external product metadata; database is the app’s persisted evidence/workflow data.

7. Methodology and algorithms

- STAC search → temporal pair → preprocessing → baseline/model → components → alert.
- Explain NDVI, `NDVI=(NIR-Red)/(NIR+Red)`, only when bands are present.
- Show: activity diagram.
- Say: “Current operational fallback is labelled spectral/NDVI baseline. A Siamese U-Net is code, not a claimed trained result.”
- Transition: “This is how the implementation is organized.”
- Question: Baseline vs ML? Baseline uses designed rules; ML learns from labelled examples and needs measured validation.

8. Technologies and implementation

- Python/FastAPI, static JS/Leaflet, SQLite/SQLAlchemy, HTTPX/CDSE, NumPy/Pillow; optional Rasterio/PyTorch/ReportLab.
- Show: directory tree and component diagram.
- Say: “I kept the initial front end lightweight and made optional heavy components explicit.”
- Transition: “Now I will show what has genuinely been verified.”
- Question: Why FastAPI? Typed APIs, automatic docs and concise Python integration.

9. Results and verification

- Dashboard/health opened; 19 tests passed; Ruff passed; real CDSE metadata query returned products.
- No live imagery-to-alert result, no trained-model metric, no SMTP delivery claim.
- Show: dashboard status and testing table.
- Say: “The strongest result is truthful observability: empty/blocked states are displayed rather than hidden.”
- Transition: “These are the remaining gaps.”
- Question: Does a unit test prove production? No, it proves a controlled behaviour only.

10. Limitations and future work

- Credentials/Rasterio/pair/model/data/evaluation remain needed.
- Clouds, seasonal changes, shadow and spatial misalignment cause false positives.
- Show: checklist P0/P1 tasks.
- Say: “My next review milestone is one reproducible real pair with provenance and human validation.”
- Transition: “I will close with the contribution.”
- Question: How reduce false positives? Cloud masks, date/pair rules, calibration, field checks and labelled evaluation.

11. Conclusion

- A traceable land-change review platform with an honest operational boundary.
- Human verification remains central.
- Show: one-sentence conclusion.
- Say: “SentinelWatch is useful because it prioritizes review while preserving where each observation and decision came from.”
- Final question: What will you do next? Configure operational path, collect lawful evidence, validate the baseline/model, and capture real case artifacts.

Delivery tips

Never say “the model detects illegal mining” unless you have trained/evaluated it and even then say “potential change.” If asked about an incomplete item, say: “The interface and service contract are implemented and tested with controlled inputs; I have not yet verified the external end-to-end integration, so I would not claim it as complete.” Keep the review focused on evidence, not promises.

SentinelWatch Viva Questions and Answers

1. Introduction and problem

1. What is SentinelWatch? A web system that helps reviewers find potential land-surface change from satellite observations.
2. What problem does it solve? It prioritizes large or remote areas that are difficult to inspect repeatedly on the ground.
3. Does it prove illegal activity? No. It produces a lead for human verification, not a legal finding.
4. Who would use it? Environmental, forestry, planning, research or compliance reviewers.

2. Architecture

5. What are the main layers? Browser dashboard, FastAPI backend, service layer, SQLite database and external CDSE services.
6. Why is there a backend? It validates input, controls access to services, persists evidence and runs longer monitoring work.
7. How is long work handled? A check-now request creates a monitoring run and starts a FastAPI background task; optional scheduling code also exists.
8. What is external? CDSE STAC, optional CDSE Process API, map tiles/CDN assets and optional SMTP.

3. Modules and data flow

9. What is an AOI? Area of Interest: the map geometry to monitor.
10. What is a monitoring run? A persistent record of one attempt to search/process a particular AOI.
11. What is a priority candidate? A preliminary grid cell ranked by metadata availability; it is not a detected incident.
12. What is the flow after Check now? Create run, search metadata, choose a pair, process only if prerequisites exist, then persist analysis/findings/alerts.

4. Frontend and backend

13. Which frontend technology is used? Server-served HTML, CSS and vanilla JavaScript; Leaflet is used for map functionality when loaded.
14. Which backend framework is used? FastAPI running with Uvicorn.
15. How does the UI communicate? JavaScript `fetch` calls JSON API endpoints.
16. What is the API documentation URL? `/docs` on the running local server.

5. APIs and external integrations

17. What is STAC? SpatioTemporal Asset Catalog, a standard API pattern for searching geospatial products by place and time.
18. How does the project find observations? It posts a search to the configured Copernicus Data Space STAC endpoint.
19. Catalogue versus application database? Catalogue is the provider’s external product index; SQLite stores the app’s selected/provenance/workflow records.
20. What needs credentials? The source separates public metadata search from authenticated imagery Process API retrieval.

6. Database and persistence

21. Which database is used? SQLite through SQLAlchemy.

22. Which important entities exist? MonitoringArea, SatelliteAcquisition, MonitoringRun, AnalysisRecord, Case, Finding, Alert, Department and NotificationAttempt.
23. How do you know it persists? The app uses `instance/sentinelwatch.db`; API/service tests verify persisted records. For a demo, restart and re-query an intentionally created record.
24. What is a migration limitation? The project has lightweight additive SQLite migrations, not a fully managed Alembic migration history.

7. Satellite imagery and remote sensing

25. Why is satellite imagery useful? It gives repeated wide-area observations with an acquisition date and geographic footprint.
26. What is acquisition date? The time the satellite measured the scene; it differs from when the software retrieves it.
27. Why are clouds a problem? Clouds and shadows can look like surface change or hide the ground.
28. What is NDVI? A vegetation indicator: `(NIR - Red)/(NIR + Red)`. It needs red and near-infrared bands and must be interpreted carefully.

8. Change detection

29. How does the baseline compare images? It normalises illumination, combines RGB/vegetation differences into a probability-like map, thresholds, cleans it and groups connected pixels.
30. How is changed area calculated? Number of changed pixels times real pixel area from geospatial metadata.
31. Why preprocess? Two images must align and unreliable cloud/shadow pixels must be masked before comparison.
32. What causes false positives? Seasonality, crops, clouds, shadows, atmosphere, alignment and sensor differences.

9. Machine learning

33. What is a Siamese network here? Two images pass through shared encoder weights so their differences can be used to predict change.
34. What is a U-Net? An encoder-decoder network that produces a pixel-level output mask.
35. Is the model trained now? No validated checkpoint or held-out evaluation is registered; do not claim model accuracy.
36. Baseline versus model? The baseline uses designed image rules; the model learns from labelled pairs and must be trained/evaluated.
37. What metrics would you use? Precision, recall, F1 and IoU on a held-out labelled set, but only after actual evaluation.

10. Alerts and departments

38. What is a priority score? A heuristic ranking based on confidence, persistence and area; it is not a probability of crime.
39. How does an alert reach a department? An alert can be assigned through the API; a notification attempt is recorded and SMTP is used only if configured.
40. How are duplicate alerts controlled? The workflow uses persisted case/finding/alert state and creation logic rather than sending a new alert for every screen refresh.
41. Can email delivery be claimed? Not in this local configuration; SMTP was not configured/tested.

11. Testing and results

42. How have you tested it? `pytest -q` passed 19 tests and Ruff passed; tests cover API, analysis, provider parsing, monitoring states, operations and reporting contracts.
43. Was real satellite data accessed? A real CDSE metadata search returned products. Imagery retrieval/analysis was not verified locally.

44. Why is a unit test not enough? It uses controlled/mocked dependencies and does not prove credentialed external or real-raster behaviour.

12. Limitations, security and future work

45. How are secrets handled? Configuration comes from environment variables; documentation must never contain real tokens or passwords.
46. What are the biggest current limitations? Missing imagery credentials/Rasterio, no trained model/dataset/metrics, and no live alert/PDF/email evidence.
47. What is the P0 next step? Configure lawful CDSE access and run one reproducible AOI through the real pair/preprocessing path.
48. How would you improve scale? Use durable workers, PostgreSQL/PostGIS, object storage, user roles and model monitoring.
49. How do you distinguish demo from live? `DEMO_MODE` and `demo_seed.py` label synthetic data; normal UI excludes synthetic test fixtures.
50. What must you verify before saying the project is complete? Real pair retrieval, preprocessing, measured analysis quality, generated case/alert/report, and any notification claim.

For difficult answers, verify the current dashboard status and the exact test output before the viva; configurations can change.

SentinelWatch Live Demonstration Guide

Before the demo

Run these commands in the project folder. Use the interpreter that has the dependencies installed.

```
python -m pip install -r requirements-core.txt
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000/`, then verify `http://127.0.0.1:8000/api/health` and OpenAPI at `http://127.0.0.1:8000/docs`. The application initializes its SQLite schema at startup. Do not copy real values into `.env.example`; make a local `.env` with CDSE/SMTP settings only when authorized.

Safe core demonstration

1. Show the dashboard title and state the boundary: it detects potential physical change, not illegality.
2. Show Live data status. Explain that `connected (sqlite)` means the local database is reachable; it does not prove satellite imagery has been downloaded.
3. Select Add monitoring area and explain AOI input/drawing. Create a small real/public area only if the reviewer permits storing it locally; otherwise show the form without submitting.
4. Show Discover priority areas. Explain it queries catalogue availability over grid cells and marks results as preliminary candidates, not incidents.
5. If CDSE metadata access is available, submit an AOI and choose Check now. The API initially returns a monitoring-run ID because processing is background work. Then refresh area/run data and inspect acquisition/provenance fields.
6. Show `/api/system/status` or dashboard system status. Explain every warning honestly—credentials, Rasterio, scheduler and model status.
7. If a real analysis completes, inspect its case/finding/alert, assign a department, inspect notification attempts, then use the case report route. Say exactly which step supplied each item.

Preparing the optional operational path

1. Install the additional prerequisites only if you need them:

```
python -m pip install -r requirements-operational.txt
```

2. Obtain authorized CDSE OAuth configuration and put it in local environment variables documented by `.env.example`.
3. Restart the service and confirm that system status no longer says imagery credentials/Rasterio are unavailable.
4. Choose a small AOI and dates with a reasonable cloud cover. A catalogue result is not automatically a usable pair; inspect date spacing and quality.
5. Use `POST /api/areas/{aoi_id}/check-now` via the dashboard or API docs. Preserve the run ID, provider product IDs and returned status for your evidence.

Alerts, departments and reports

- Add a review department through the dashboard/API.
- Assign a genuine alert only after one exists. Inspect `GET /api/alerts/{alert_id}/notifications` to show whether the notification was merely recorded, not configured, sent or failed.
- Generate the case report only for a real stored case. If ReportLab is unavailable, say the report module is optional and unverified, not that the report was generated.

If external services fail

- No catalogue response: show the error/refresh status; do not retry indefinitely or invent observations.
- No suitable pair: explain cloud/date constraints and show that the run ends without an analysis.

- No CDSE OAuth/Rasterio: demonstrate metadata/UI/API/SQLite modules and point to the transparent blocked status.
- No trained model: show `NOT_TRAINED`; describe baseline code as the current analysis method, not AI performance.
- No SMTP: show recorded notification status and say delivery was not demonstrated.

Backup 2-minute sequence

Start the app, open the dashboard, show `/api/health`, status cards, model governance, and `/docs`. Explain the as-built architecture and run `python -m pytest -q` plus `python -m ruff check .` beforehand. This demonstrates verified platform behaviour without pretending an external imagery pipeline is live.

Demo data

`python -m app.demo_seed` can create synthetic fixtures only if you explicitly choose to use it. Set `DEMO_MODE=true` only for that labelled demonstration, say “synthetic demo data” on every relevant slide, and never mix it with claims about live satellite results.

SentinelWatch Completion Checklist

Requirement: App startup/health
Current status: Complete
Evidence: local `/api/health` returned 200
Priority: P0
Next action: repeat before review

Requirement: Frontend/backend connection
Current status: Complete
Evidence: dashboard loaded live API state
Priority: P0
Next action: retain screenshot after final setup

Requirement: Persistent database
Current status: Complete for local SQLite
Evidence: status says connected; tests pass
Priority: P0
Next action: restart test with a test record

Requirement: AOI CRUD/run API
Current status: Complete/tested
Evidence: routes and tests
Priority: P0
Next action: use a real small AOI

Requirement: Automatic candidate discovery
Current status: Complete/tested with controlled provider
Evidence: service/API tests
Priority: P1
Next action: perform/capture real catalogue discovery

Requirement: Real catalogue search
Current status: Complete for metadata
Evidence: live CDSE query returned 10 products
Priority: P0
Next action: record final AOI query

Requirement: Imagery retrieval
Current status: Not verified
Evidence: needs OAuth credentials
Priority: P0
Next action: configure CDSE Process API

Requirement: Raster preprocessing
Current status: Implemented, not verified locally
Evidence: `geospatial.py`; Rasterio absent
Priority: P0
Next action: install/test Rasterio

Requirement: Baseline change algorithm
Current status: Complete/tested in controlled arrays
Evidence: analysis tests
Priority: P1
Next action: validate against real labelled pair

Requirement: Siamese model
Current status: Partial
Evidence: model/training code; NOT_TRAINED
Priority: P1
Next action: dataset, train, checkpoint, metrics

Requirement: Map visualization
Current status: Partial
Evidence: Leaflet UI code
Priority: P1
Next action: capture a real AOI/map

Requirement: Case/alert generation
Current status: Complete/tested contract
Evidence: services/API tests
Priority: P1
Next action: obtain real result

Requirement: Department routing
Current status: Complete/tested contract
Evidence: operations API tests
Priority: P1
Next action: assign real alert

Requirement: Notification delivery
Current status: Implemented, unverified
Evidence: notification status code; SMTP absent
Priority: P2
Next action: configure/test SMTP

Requirement: PDF report
Current status: Implemented, unverified
Evidence: report route/service
Priority: P1
Next action: generate/open genuine PDF

Requirement: Scheduler/background robustness
Current status: Partial
Evidence: background task; scheduler disabled
Priority: P2
Next action: enable/verify durable execution

Requirement: Automated tests
Current status: Complete
Evidence: 19 passed
Priority: P0
Next action: rerun after changes

Requirement: Lint
Current status: Complete
Evidence: Ruff passed
Priority: P0
Next action: rerun after changes

Requirement: Real screenshots
Current status: Incomplete
Evidence: live dashboard visually inspected; no export artifact saved
Priority: P1
Next action: capture dashboard/AOI/result/alert/report after P0 path works

Requirement: Review documentation
Current status: Complete
Evidence: `docs/` package
Priority: P0
Next action: rehearse from speaking guide

Requirement: Presentation/viva prep
Current status: Complete
Evidence: speaking guide/question bank
Priority: P0
Next action: practise honest answers

Status meanings: Complete has direct code plus test/execution evidence; Partial has incomplete or unverified important behaviour; Not verified is deliberately not claimed complete.

Diagram Sources

These are the Mermaid sources used by the review. They are included in full so the diagrams can be regenerated in a Mermaid-compatible renderer. The source files remain in docs/diagrams/.

Activity

```
flowchart TD
  A([Start]) --> B[Define or approve area]
  B --> C[Run monitor now or scheduler]
  C --> D[Search CDSE STAC]
  D --> E{Enough suitable observations?}
  E -- No --> F[Store status / explain no pair]
  F --> Z([End])
  E -- Yes --> G{Credentials + rasterio available?}
  G -- No --> H[Record blocked run]
  H --> Z
  G -- Yes --> I[Retrieve and preprocess image pair]
  I --> J[Baseline or model analysis]
  J --> K{Significant change?}
  K -- No --> L[Store analysis with no alert]
  K -- Yes --> M[Store finding, case and alert]
  L --> Z
  M --> N[Optional department routing / email]
  N --> Z
```

Architecture

```
flowchart LR
  U[Reviewer / Operator] --> UI[Dashboard\nHTML + JS + Leaflet]
  UI --> API[FastAPI application]
  API --> DB[(SQLite + SQLAlchemy)]
  API --> MON[Monitoring service\nbackground task]
  MON --> STAC[CDSE STAC catalogue\nexternal]
  MON -. credentials required .-> PROC[CDSE Process API\nexternal]
  MON -. rasterio required .-> GEO[Geospatial preprocessing]
  GEO --> ANALYSE[Baseline analysis\nor optional Siamese U-Net]
  ANALYSE --> DB
  ANALYSE --> ALERT[Alerting + priority]
  ALERT --> DB
  ALERT -. optional SMTP .-> MAIL[Configured email recipient]
  API --> PDF[Case PDF reporting]
  PDF --> U
```

Component

```
flowchart LR
  FE[Static frontend\nindex.html / app.js / CSS] --> BE[FastAPI routers\nmain, monitoring, operations]
  BE --> SVC[Services\nmonitoring, analysis, alerting, reporting]
  SVC --> SAT[Satellite adapter\ncopernicus.py]
  SVC --> ML[ML package\nsiamese_unet + inference]
  SVC --> GEO[Geospatial service]
  BE --> ORM[SQLAlchemy models]
  ORM --> SQL[(SQLite)]
  SAT --> CDSE[CDSE endpoints]
  SVC --> SMTP[Optional SMTP]
```

Deployment

```
flowchart TB
  B[User browser\nLeaflet CDN + OpenStreetMap] --> H[Host machine / container]
  subgraph H[Host machine / Docker container]
    A[Uvicorn + FastAPI]
    S[(SQLite file\ninstance/sentinelwatch.db)]
    F[storage/ previews and reports]
    A <--> S
    A <--> F
  end
  A --> C[Internet: CDSE STAC]
  A -. only configured .-> P[Internet: CDSE Process API]
  A -. only configured .-> M[SMTP server]
```

Dfd Level 0

```
flowchart LR
  USER[Operator / reviewer] -->|areas, discovery requests, reviews| SW((SentinelWatch))
  SW -->|dashboard, cases, alerts, reports| USER
  CDSE[Copernicus Data Space Ecosystem] -->|catalogue metadata; imagery when configured| SW
  SW -->|STAC search / Process API request| CDSE
  SW -. optional notification .-> EMAIL[Configured email recipient]
```

Dfd Level 1

```
flowchart TB
  U[Operator] --> P1[1. Manage area / discover candidate]
  P1 --> D1[(SQLite database)]
  P1 --> P2[2. Queue monitoring run]
  P2 --> P3[3. Search CDSE catalogue]
  P3 <--> E1[CDSE STAC]
  P3 --> D1
  P3 --> P4[4. Select temporal pair]
  P4 -. configured credentials + rasterio .-> P5[5. Retrieve and preprocess imagery]
  P5 --> P6[6. Change analysis]
  P6 --> D1
  P6 --> P7[7. Create finding / alert]
  P7 --> D1
  P7 -. optional SMTP .-> E2[Email service]
  D1 --> P8[8. Dashboard / case report]
  P8 --> U
```

Dfd Level 2

```
flowchart TB
  A[AOI + monitoring run] --> S[Search STAC metadata]
  S --> C[Persist acquisitions / catalogue refresh]
  C --> P[Choose before and after observations]
  P --> Q{Credentials and rasterio available?}
  Q -- No --> X[Record blocked / failed run; no false analysis]
  Q -- Yes --> R[Request six-band GeoTIFF]
  R --> G[Align rasters, mask cloud/shadow, calculate NDVI]
  G --> M{Validated Siamese checkpoint?}
  M -- Yes --> ML[Siamese U-Net probability map]
  M -- No --> B[Spectral baseline probability map]
  ML --> T[Threshold, morphology, connected components]
  B --> T
  T --> F[Classify region, calculate area, save finding]
  F --> L[Create deduplicated alert when threshold is met]
```

Er Diagram

```
erDiagram
  MonitoringArea ||--o{ AOIAcquisition : links
  SatelliteAcquisition ||--o{ AOIAcquisition : links
  MonitoringArea ||--o{ MonitoringRun : schedules
  MonitoringArea ||--o{ AnalysisRecord : analyses
  AnalysisRecord ||--o{ Case : creates
  Case ||--o{ Finding : contains
  Case ||--o{ Alert : contains
  Case ||--o{ CaseReview : receives
  Department ||--o{ Alert : assigned_to
  Alert ||--o{ NotificationAttempt : records
  MonitoringArea ||--o{ PriorityCandidate : approved_from
  MonitoringArea {
    string id PK
    string name
    json geometry
    boolean is_active
  }
  SatelliteAcquisition {
    string id PK
    string provider_product_id
    datetime acquired_at
    float cloud_cover
  }
  AnalysisRecord {
    string id PK
    string method
    string status
    datetime created_at
  }
  Case {
    string id PK
    string status
    string title
  }
  Finding {
    string id PK
    string change_type
    float confidence
    float area_m2
  }
  Alert {
    string id PK
    string severity
    float priority_score
    string status
  }
```

Sequence

```
sequenceDiagram
  actor O as Operator
  participant UI as Browser dashboard
  participant API as FastAPI
  participant DB as SQLite
  participant MON as Monitoring service
  participant CDSE as CDSE STAC/Process API
  participant AN as Analysis service
  participant AL as Alert service
  O->>UI: Click Check now
  UI->>API: POST /api/areas/{id}/check-now
  API->>DB: create MonitoringRun
  API-->>UI: 202 Accepted + run id
  API->>MON: background execution
  MON->>CDSE: STAC search
  CDSE-->>MON: acquisition metadata
  MON->>DB: save metadata/run status
  opt credentials + rasterio installed
    MON->>CDSE: request imagery
    MON->>AN: preprocess and compare pair
    AN-->>MON: analysis/finding candidates
    MON->>AL: create alerts if threshold met
    AL->>DB: persist case, finding, alert
  end
  UI->>API: read areas/cases/alerts
  API->>DB: query persisted records
  API-->>UI: JSON dashboard data
```

Use Case

```
flowchart LR
  O([Operator]) --> A1[Create / pause monitoring area]
  O --> A2[Discover / approve candidate area]
  O --> A3[Start check now]
  O --> A4[Inspect cases and alerts]
  O --> A5[Assign department / review case]
  O --> A6[Download case report]
  CDSE([CDSE service]) --> A7[Provide catalogue metadata]
  CDSE --> A8[Provide raster imagery when authenticated]
  MAIL([Configured SMTP recipient]) <-->|Optional email| A9[Deliver notification]
  A3 --> A7
  A3 -. optional .-> A8
```